

根据你提供的实验需求，我为你整理了一份完整的实验报告。这份报告结合了你提供的 PDF 内容和实验操作要点。

实验报告：Makefile 编译工具与构建自动化

实验目的：掌握 Makefile 的基本编写方法，理解编译、链接的过程，并熟悉多目录、多层次的项目构建管理方式。

一、实验环境

•

操作系统：Linux (Ubuntu/CentOS)

•

编译器：GCC (GNU Compiler Collection)

•

构建工具：GNU Make

二、题 1：基础 Makefile 编写

1. 实验需求

- 编译 `main.c` 和 `utils.c` 以及头文件 `utils.h`。
- 生成可执行文件 `myapp`。
- 实现 `clean`（清理）和 `rebuild`（重构）功能。
- 使用变量定义编译器和参数。

2. 代码实现

```
# 变量定义
CC = gcc
CFLAGS = -Wall -O2
TARGET = myapp
OBJS = main.o utils.o

# 核心构建目标
$(TARGET): $(OBJS)
    @echo "Building $(TARGET)..."
    $(CC) $(OBJS) -o $(TARGET)

# 编译 main.o
main.o: main.c utils.h
    @echo "Compiling main.o..."
    $(CC) $(CFLAGS) -c main.c

# 编译 utils.o
utils.o: utils.c utils.h
    @echo "Compiling utils.o..."
    $(CC) $(CFLAGS) -c utils.c

# 清理功能
clean:
    @echo "Removing object files..."
    rm -f $(OBJS) $(TARGET)

# 重新构建
rebuild: clean $(TARGET)

.PHONY: clean rebuild
```

3. 运行测试

- 执行 `make`：终端会依次显示 `Compiling...` 和 `Building...`。
- 执行 `./myapp`：程序正常运行并输出预期结果。
- 执行 `make clean`：成功删除生成的 `.o` 和可执行文件。

三、 题 2： Makefile 逻辑分析

1. Makefile 1： 顶层环境配置

-

环境变量导出：通过 `export` 关键字将当前路径 (`Top`)、源码路径 (`Src`) 等定义为全局环境变量，供后续递归调用的子 Makefile 使用。

-

递归编译：使用 `@$(MAKE) -C $(Src)` 指令进入 `src` 目录执行编译任务，这是大型项目常用的分层构建方式。

-

安装与清理：`install` 将生成的目标文件拷贝到项目根目录；`clean` 通过 `-rm -rf` 强制删除构建目录和测试程序，其中 `-` 符号表示忽略删除不存在文件时产生的错误。

2. Makefile 2： 目录管理与链接

-

文件归档：在主程序编译完成后，自动创建 `build` 文件夹并将所有 `.o` 文件移动其中，保持根目录整洁。

-

子目录构建：分别进入 `dir1` 和 `dir2` 进行子模块编译。

-

多对象链接：最终链接命令 `$(CC) -o $(Build)/test $(Build)/*.o ...` 汇总了所有目录下的目标文件，生成最终的可执行文件。

-

头文件依赖：通过 `main.o: $(Include)/func.h` 明确了头文件依赖关系，确保头文件修改后能触发重新编译。

四、 题 3： GCC 编译命令手动实践

通过手动输入 GCC 命令，我理解了从源代码到可执行文件的四个阶段：

阶段	命令示例	说明
预处理	<code>gcc -E main.c -o main.i</code>	处理宏定义、头文件展开

|

| **编译** | `gcc -S main.i -o main.s` | 将 C 语言翻译成汇编代码

|

| **汇编** | `gcc -c main.s -o main.o` | 将汇编代码转换为机器指令

|

| **链接** | `gcc main.o -o myapp` | 合并库函数和多个目标文件

|

实验发现与心得记录

- 1.
- 静默执行：**在 Makefile 命令前加 @ 可以隐藏指令本身，只显示 echo 的提示信息，这让编译输出非常专业。
- 增量编译：**Makefile 极大地提高了效率。如果我只修改了 utils.c ，再次 make 时，main.o 不会被重新编译，这在大型工程中能节省大量时间。
- 3.
- 目录层级：**题 2 的设计让我意识到，规范的项目结构（如源码在 src ，编译产物在 build ，头文件在 include ）是工程化的基础。
- 4. Makefile 的“坑”：**如果 Makefile 目标名与当前目录下的文件名重名（如名为 clean 的文件），必须使用 .PHONY 声明，否则 make clean 会失效。